

EDUSPACE

تحقیق جامع پخش همزمان ویدئو

Co-Watch و پخش پیش از پایان آپلود

EduSpace تحلیل فنی، محصول، بازار، ریسک، هزینه و نقشه راه مخصوص

نتیجه اصلی: قابلیت ارزشمند و شدنی است؛ اما باید با «مدت واقعاً آماده پخش»، کیفیت تطبیقی و fallback طراحی شود، نه با وعده عمومی «۵٪ فایل» یا «صفر لگ».

EduSpace مخاطب: تیم محصول، فنی و کسب و کار

نسخه: ۱.۰ | تاریخ: ۲۹ اوت ۲۰۲۶

codex/fix/camera-background-lifecycle مبنای پروژه: شاخه

وضعیت: داخلی - جهت تصمیم‌گیری

نقشه تصمیم در یک صفحه

توصیه: Shared Player و VOD آماده را ابتدا عرضه کنید؛ Progressive Partial Playback را فقط پس از Spike موفق فعال کنید؛ برای شروع فوری، مسیر زنده موقت و handoff به HLS را در نظر بگیرید.

گزینه	مزیت اصلی	محدودیت اصلی	جایگاه پیشنهادی
Managed VOD	سریع‌ترین راه به ABR/CDN	پخش معمولاً پس از upload/processing	Pilot و اثبات بازار
Self-hosted VOD	کنترل داده و vendor	عملیات رسانه و ظرفیت‌سازی سنگین	وقتی vendor مناسب نیست
Progressive ingest	پخش قبل از پایان upload	برای هر MP4 عمومی نیست؛ seek محدود	فاز آزمایشی پس از Spike
Live + handoff	شروع فوری از دستگاه میزبان	handoff وابسته به uplink تا میزبان	fallback تجربه کاربر

مبنای مقایسه: شواهد رسمی استانداردها، سرویس‌های ویدئو، رقبا و کد زنده EduSpace.

راهنمای مطالعه

- بخش‌های ۱ تا ۳: مسئله کاربر، اعتبارسنجی گزارش قبلی و بازار.
- بخش‌های ۴ تا ۶: اصول فنی، گزینه‌ها و معماری مخصوص EduSpace.
- بخش‌های ۷ تا ۹: UX، امنیت، حقوق، هزینه و ظرفیت.
- بخش‌های ۱۰ تا ۱۴: نقشه راه، KPI، تست، تصمیم‌های باز و توصیه نهایی.

پاسخ مدیریتی مستقیم

این قابلیت ارزش محصولی واقعی دارد، اما تعریف درست آن «پخش بدون لگ با هر اینترنت» نیست. تعریف قابل ساخت و قابل دفاع چنین است:

برگزارکننده فایل را با آپلود قابل ازسرگیری وارد می‌کند؛ EduSpace به محض آماده‌شدن یک بازه امن ویدئو، پخش همگام را فعال می‌کند؛ هر بیننده کیفیت متناسب با شبکه خود می‌گیرد؛ کنترل زمانی در اختیار برگزارکننده است؛ و سیستم بهروشنی نشان می‌دهد چه مقدار از فیلم واقعاً قابل پخش است.

پیشنهاد نهایی، یک راهبرد سه‌مرحله‌ای است:

1. **MVP** کم‌ریسک: ویدئوی از قبل آماده‌شده یا لینک HLS/MP4 معتبر، پلیر مشترک، کنترل میزبان، بازیابی پس از reconnect و کیفیت تطبیقی در صورت وجود HLS.
 2. نسخه تجاری قابل اتکا: آپلود مستقیم و قابل ازسرگیری، پردازش پس از تکمیل، HLS/CMAF چندکیفیتی، CDN، سیاست نگهداری و داشبورد کیفیت.
 3. پخش پیش از پایان آپلود: فقط پس از یک Spike فنی؛ با preflight سمت کلاینت، staging پایدار، انتشار سگمنت‌های کامل و محدودکردن seek به بازه آماده. برای «هر فایل دلخواه» نباید از روز اول وعده عمومی داده شود.
- اگر «شروع همین حالا» شرط اصلی جلسه باشد، مسیر ترکیبی بهترین تجربه است: فایل از دستگاه میزبان مثل Zoom به‌صورت یک contribution زنده ویدئویی پخش می‌شود، همزمان نسخه اصلی آپلود و آماده می‌شود، و بعد سیستم در یک نقطه امن به HTTP/HLS تحویل می‌دهد. این حالت شروع را سریع می‌کند، ولی تا زمان تحویل به HLS هنوز به آپلود پایدار میزبان وابسته است.

۱. مسئله کاربر و فرصت محصول

۱.۱. کار اصلی که کاربر می‌خواهد انجام دهد

- مدرس می‌خواهد یک کلیپ یا فیلم بلند را بدون اتلاف وقت کلاس پخش کند.
- برگزارکننده می‌خواهد Play/Pause/Seek برای همه هماهنگ باشد و شرکت‌کننده نتواند ناخواسته جریان کلاس را به‌هم بزند.
- بیننده با اینترنت ضعیف ترجیح می‌دهد کیفیت کمتر ببیند ولی صدا و تداوم حفظ شود.
- کاربر دیررس باید به موقعیت جاری برسد، نه اینکه فیلم را از ابتدا ببیند.
- مدیر سازمان باید روی دسترسی، مدت نگهداری، هزینه، محتوای نامناسب و مصرف پهنای‌باند کنترل داشته باشد.

۱.۲. چرا Screen Share راه‌حل کامل نیست

Screen Share و اشتراک فایل محلی، شروع سریعی دارند اما مسیر رسانه را به توان CPU و uplink میزبان گره می‌زنند. مستندات رسمی Zoom می‌گوید پلیر فایل محلی، MOV/MP4 با H.264 را مستقیماً encode و share می‌کند و برای Full Screen HD حداقل 1.5 Mbps رفت‌وبرگشت می‌خواهد؛ این روش نسبت به screen share عادی نرم‌تر است، اما همچنان یک rebroadcast زنده از دستگاه ارائه‌دهنده است ([Zoom Support](#)). Microsoft Teams و Google Meet نیز ویدئو را از مسیر ارائه صفحه/تب و صدای سیستم می‌فرستند؛ خود Google ناپایداری پهنای‌باند و latency را علت مهم افت کیفیت می‌داند ([Google Meet Help](#)، [Microsoft Support](#)).

در مقابل، HTTP streaming با HLS/CMAF می‌تواند delivery را از RTC جدا کند: هر بیننده segmentها را مستقل می‌گیرد، CDN آنها را cache می‌کند و ABR کیفیت را برای همان بیننده تغییر می‌دهد. این معماری «ریسک مشترک uplink میزبان» را کم می‌کند، اما upload، packaging و پردازش و آماده‌بودن asset را به سیستم اضافه می‌کند.

۱.۳. ارزش پیشنهادی درست

پیام پیشنهادی:

«پخش همزمان را پیش از پایان کامل آماده‌سازی شروع کنید—به محض اینکه بخش کافی ویدئو قابل پخش باشد. کیفیت برای اینترنت هر بیننده تطبیق می‌یابد و کنترل در اختیار برگزارکننده می‌ماند.»

افشای کوتاه لازم:

«زمان شروع و کیفیت به فرمت فایل، سرعت آپلود، توان پردازش و اینترنت هر بیننده بستگی دارد.»

عبارت‌های نامناسب برای مارکتینگ: «بدون لگ»، «صفر تأخیر»، «همیشه فوری»، «با هر اینترنتی» و «با رسیدن ۵٪ فایل حتماً پخش می‌شود».

۲. اعتبارسنجی گزارش قبلی

گزارش پیوست جهت کلی مفیدی دارد: جداسازی media delivery از control plane، استفاده از upload قابل ازسرگیری، HLS و همگام‌سازی از طریق LiveKit. با این حال چند ادعای آن باید اصلاح شود.

۲.۱. ادعاهای تأییدشده

- MP4 معمولی غالباً metadata سراسری خود را در انتهای فایل می‌نویسد؛ faststart آن را پس از تکمیل به ابتدای فایل جابه‌جا می‌کند. fragmented MP4 برای خواندن تدریجی مناسب‌تر است ([FFmpeg Formats](#)).
- MSE برای ISO-BMFF به initialization segment شامل moov + ftyp و سپس media segmentهای معتبر نیاز دارد ([W3C ISO BMFF Byte Stream Format](#)).
- HLS می‌تواند playlist رویدادی در حال رشد داشته باشد و segmentهای کامل را به انتهای آن اضافه کند (RFC 8216).
- tus برای upload قابل ازسرگیری مناسب است و LiveKit data packets برای فرمان‌های سبک زمان‌بندی مناسب‌اند ([tus Protocol](#)، [LiveKit Data Packets](#)).

۲.۲. ادعاهای اصلاح‌شده یا ردشده

- «۹۰٪ فایل‌ها moov انتهایی دارند»: منبع معتبری برای عدد ۹۰٪ پیدا نشد. فقط عبارت «معمولاً» در مستندات FFmpeg پشتیبانی می‌شود.
- «وقتی ۵۰ MB یا ۵٪ رسید Play فعال شود»: درصد بایت با دقیقه قابل پخش برابر نیست؛ VBR، جایگاه metadata، keyframe و codec تعیین‌کننده‌اند. معیار باید published_duration و buffer امن باشد.
- «tus/S3 multipart ذاتاً streamable است»: resumability با streamability یکی نیست. S3 object پس از CompleteMultipartUpload ساخته می‌شود؛ parts upload همان object قابل GET نیستند ([AWS S3 Multipart Upload](#)).

- «FIFO» مستقیم راه استاندارد طلایی است: «FIFO non-seekingable و ephemeral است و «resume، retry، duplicate، gap و restart را حل نمی‌کند. منبع حقیقت باید staging پایدار با committed offset باشد.
- «فرمان نمونه LL-HLS تولید می‌کند»: 3- hls_time با TS معمولی، LL-HLS نیست. Partial به LL-HLS Segment و سازوکارهایی مانند EXT-X-PART و blocking reload نیاز دارد ([Apple LL-HLS](#)).
- «codec copy» راه عمومی ABR است: «stream copy سریع است، اما ladder چندکیفیتی، «resize، normalization و keyframe alignment معمولاً به transcode نیاز دارند ([FFmpeg Streamcopy](#)).
- «zero wait / zero lag»: شبکه، segment production، encode، device و startup buffer تأخیر اجتناب‌ناپذیر دارند. باید SLO اندازه‌گیری شده تعریف شود.
- «زمان‌بندی ۷ هفته قطعی است»: بدون Spike روی corpus فایل واقعی، تست بار و انتخاب build-vs-buy قابل تعهد نیست.

۳. بنچ‌مارک بازار

در هفت محصول بررسی شده، هیچ مستند رسمی مثبتی برای «آپلود فایل به خود پلتفرم و پخش همان دارایی پیش از پایان همان upload» پیدا نشد. این یک نتیجه از اسناد بررسی شده است، نه اثبات نبود قابلیت در همه نسخه‌ها.

۳.۱. الگوی A: بازپخش زنده از دستگاه میزبان

Zoom. فایل MOV/MP4 با H.264 را در پلیر داخلی باز می‌کند؛ کنترل‌ها در دست sharer است و encode مستقیم فایل از screen capture کم‌هزینه‌تر است. این upload-to-library نیست ([Zoom Support](#)).

Microsoft Teams. ویدئو از طریق share screen/window و Include sound ارائه می‌شود؛ Teams برای محتوای متحرک frame rate را ترجیح می‌دهد ([Microsoft Support](#)).

Google Meet. ارائه تب/پنجره/صفحه و صدای تب یا سیستم را فراهم می‌کند؛ asset ویدئویی upload شده و server-side shared player در راهنمای رسمی مشاهده نشد ([Google Meet Help](#)).

نتیجه: شروع سریع و UX آشنا، اما quality و continuity تا حد زیادی تابع دستگاه و uplink ارائه‌دهنده است.

۳.۲. الگوی B: منبع مستقیم برای هر بیننده، همگام‌سازی فقط برای state

BigBlueButton. presenter یک URL از YouTube/Vimeo و منابع دیگر یا direct MP4/MP3 می‌دهد؛ هر کاربر رسانه را از منبع می‌گیرد و BBB Play/Pause/Seek را هماهنگ می‌کند. ویدئوی خارجی در recording پردازش شده ظاهر نمی‌شود ([BigBlueButton Docs](#)).

Teleparty و **Watch2Gether**. بر هماهنگ‌سازی state روی سرویس‌های بیرونی تکیه می‌کنند. Teleparty به account و دسترسی منطقه‌ای هر کاربر نیاز دارد ([Teleparty Support](#)). این مدل زیرساخت media را سبک می‌کند، اما buffering و availability را به منبع خارجی می‌سپارد.

۳.۳. الگوی C: asset آپلود شده و buffer شده

Adobe Connect. MP4 مستقیماً داخل Share Pod آپلود می‌شود و host/presenter کنترل play/seek/pause دارد. Adobe توصیه می‌کند bitrate تا 2 Mbps باشد، چون هر participant به پهنای باند بالاتر از bitrate فایل نیاز دارد ([Adobe Connect](#)، مارس ۲۰۲۵). مستند رسمی workflow را upload و سپس Share نشان می‌دهد، نه پخش در حین upload.

۳.۴. فضای تمایز EduSpace

تمایز واقعی می‌تواند ترکیب این سه مزیت باشد:

- UX شروع سریع شبیه فایل محلی Zoom؛
- delivery مستقل و sync شبیه BBB؛
- asset خصوصی، کنترل دسترسی و buffer شبیه Adobe Connect؛
- همراه با readiness قابل مشاهده و محدودیت صادقانه روی بازه آماده.

۴. اصول فنی تعیین کننده

۴.۱. upload progress با playable progress فرق دارد

برای فعال شدن Play باید این ها آماده باشند:

1. container و codec شناسایی و مجاز باشند؛
 2. initialization segment معتبر وجود داشته باشد؛
 3. حداقل چند segment کامل و قابل decode منتشر شده باشد؛
 4. segment از keyframe مناسب شروع شود؛
 5. سرعت ingest/encode از مصرف rendition انتخابی با حاشیه امن عقب نماند.
- پس UI باید سه شاخص جدا نشان دهد: «آپلود»، «پردازش»، و «مدت آماده پخش». نمونه: «آپلود ۳۸٪؛ ۱۲ دقیقه آماده پخش؛ کیفیت ۳۶۰p و ۷۲۰p آماده».

۴.۲. segment باید قبل از advertise شدن کامل باشد

در HLS عادی، URI وارد شده در playlist باید فوراً قابل دانلود باشد و playlist باید atomically به روزرسانی شود (RFC 8216). الگوی امن:

1. worker segment را کامل تولید می کند؛
 2. checksum و مدت آن ثبت می شود؛
 3. segment immutable در object storage قرار می گیرد؛
 4. سپس manifest با نسخه جدید منتشر می شود؛
 5. CDN manifest کوتاه عمر و segment ها را بلند عمر cache می کند.
- Redis محل segment نیست؛ برای state، lock، progress و event مناسب است. `dev/shm/` نیز منبع حقیقت نیست؛ ظرفیت محدود و ماهیت volatile دارد.

۴.۳. upload protocol با media ingest protocol فرق دارد

- tus باید مسئول offset، resume و retry باشد. pipeline media باید فقط contiguous committed prefix یا objectهای segment مستقل را بخواند. در حالت parallel upload، chunkها ممکن است خارج از ترتیب برسند؛ آن ها نباید مستقیم وارد FIFO شوند. staging پیشنهادی:
- durable append-only spool یا فایل sparse کنترل شده؛
 - جدول offsetهای commit شده و checksum؛

- worker idempotent با checkpoint
- cancellation، timeout و cleanup
- original upload مستقل از rendition output.

۴.۴. transmux و transcode

Fast path: اگر profile، timestamps، H.264/AAC و GOP سازگار باشند، transmux به CMAF/HLS بدون encode مجدد می‌تواند سریع باشد.

Fallback: برای codec ناسازگار، resolution بالا، audio نامعتبر، VFR مشکل‌ساز یا transcode ABR ladder، ladder اولیه می‌تواند 360p و 720p باشد؛ فقط با plan/سیاست مناسب.

اصل عملی: ابتدا یک rendition کم‌حجم را playable کن، سپس کیفیت‌های بالاتر را اضافه کن. Cloudflare Stream نیز ویدئوی ready را قابل پخش می‌داند در حالی که بعضی quality levelها ممکن است هنوز encode شوند ([Cloudflare Stream FAQ](#)).

۴.۵. LL-HLS لازم نیست

محتوا «فایل انتخاب‌شده» است، نه دوربین live. هدف، کم‌کردن انتظار upload است؛ نه رساندن live edge به زیر چند ثانیه. HLS/CMAF event-style با segmentهای 2 تا 4 ثانیه برای MVP ساده‌تر و CDN-friendly است. LL-HLS پیچیدگی origin/CDN/player را زیاد می‌کند و فقط وقتی metricهای واقعی نیاز را ثابت کردند ارزش دارد.

۵. گزینه‌های معماری

گزینه ۱: سرویس ویدئوی مدیریت‌شده

نمونه‌ها: Cloudflare Stream و Mux.

مزایا:

- upload مستقیم و signed playback، player/CDN، ABR، encode، resumable و analytics
- کمترین زمان عملیات و ریسک codec
- مناسب برای MVP کیفیت بالا بعد از آماده‌شدن asset.

محدودیت:

- workflow استاندارد VOD هر دو، upload complete و سپس processing/ready است؛ Mux صریحاً video.asset.ready را زمان شروع playback می‌داند ([Mux Uploader](#)). Cloudflare نیز tus را برای resume ارائه می‌دهد، نه پخش asset ناقص ([Cloudflare Resumable Upload](#)).
- وابستگی فروشنده، availability منطقه‌ای، پرداخت ارزی و الزامات حقوقی باید جداگانه بررسی شود.

هزینه مرجع در تاریخ تحقیق: Cloudflare Stream برای ۱۰۰۰ دقیقه ذخیره ۵ دلار و برای ۱۰۰۰ دقیقه delivery یک دلار اعلام می‌کند؛ ingress و encoding را رایگان می‌داند ([Cloudflare Pricing](#)، آوریل ۲۰۲۶). Mux قیمت را بر input/storage/delivery minute و resolution بنا می‌کند و ۱۰۰ هزار دقیقه delivery ماهانه را در پلن‌های اعلام‌شده رایگان نشان می‌دهد ([Mux Pricing](#)). این قیمت‌ها فقط baseline هستند و برای ایران، مالیات، قرارداد، ارز و availability باید در زمان خرید دوباره بررسی شوند.

گزینه ۲: self-hosted upload-complete سپس HLS/ABR

مزایا:

- کنترل داده، retention و هزینه؛
 - امکان استفاده از S3-compatible storage و CDN انتخابی؛
 - reuse بخشی از Celery/FFmpeg موجود.
- محدودیت:
- مسئولیت کامل observability، retries، codec matrix، sandbox، ظرفیت manifest، GPU/CPU، correctness و delivery؛
 - زمان و ریسک بیشتر از managed service.
- برای production reverse proxy/deployment در EduSpace، repository موجود نیست و storage/CDN فقط به صورت اختیاری برای recording دیده می شود. این گزینه قبل از rollout عمومی به media queue و worker جدا، lifecycle policy، object storage و CDN نیاز دارد.

گزینه ۳: progressive ingest واقعی

مسیر:

client preflight → resumable upload → durable spool → conditional transmux/transcode → immutable CMAF segments → event playlist → CDN/player

مزایا:

- نزدیکترین حالت به خواسته اصلی؛
 - شروع پخش بعد از buffer امن، قبل از upload کامل؛
 - امکان نمایش playable frontier.
- محدودیت:
- برای MP4 با moov انتهایی، worker ممکن است تا tail نتواند demux کند؛
 - backpressure، resume/out-of-order و restart پیچیده اند؛
 - seek فقط تا segment منتشر شده ممکن است؛
 - ABR زود هنگام CPU/GPU بیشتری می خواهد.
- این گزینه باید feature-flagged و ابتدا فقط برای profile های سازگار فعال شود.

گزینه ۴: شروع فوری ترکیبی و handoff

1. میزبان فایل را انتخاب می کند.
2. آپ فایل را local decode/encode و مثل یک media contribution زنده از LiveKit می فرستد.
3. همزمان original به VOD upload pipeline می شود.
4. وقتی rendition HTTP timestamp مشترک رسید، میزبان در pause یا keyframe امن handoff می کند.

این گزینه از نظر UX قوی است و مشکل «منتظر upload کامل نباش» را فوراً حل می‌کند، اما تا handoff به uplink میزبان وابسته است. پیاده‌سازی handoff باید از ابتدا به‌عنوان state transition طراحی شود، نه تعویض پنهانی و پرریسک وسط playback.

۶. معماری پیشنهادی برای EduSpace

۶.۱. تصمیم پیشنهادی

راه اصلی: گزینه ۱ یا ۲ برای VOD آماده + sync مستقل.

راه شروع فوری: گزینه ۴ در desktop به‌عنوان fallback.

راه آزمایشی: گزینه ۳ برای فایل‌های سازگار، بعد از Spike.

انتخاب managed یا self-hosted باید با یک آزمایش هزینه/کیفیت انجام شود. اگر vendor access از شبکه‌های هدف یا پرداخت سازمانی مانع باشد، self-hosted منطقی می‌شود؛ در غیر این صورت managed service برای اثبات بازار سریع‌تر است.

۶.۲. انطباق با معماری فعلی

EduSpace اکنون این دارایی‌های قابل reuse را دارد:

- LiveKit و reliable/lossy packet های data
- الگوی sync موجود برای Presentation با start/page/stop
- RBAC اتاق و مجوز host/co-host/participant
- مدل lifecycle امن برای سندهای upload شده؛
- Celery/Redis و FFmpeg برای recording
- storage محلی و S3 اختیاری برای recording.

شکاف‌ها:

- entity و state machine مخصوص SharedVideoAsset و SharedPlaybackSession؛
- tus/direct upload و quota
- media probe/sandbox/transcode queue
- HLS/CMAF player (کتابخانه‌ای مثل hls.js هنوز در dependencies نیست)؛
- production object storage + CDN
- قرارداد مشترک و versioned برای realtime messages
- analytics کیفیت playback و cost guardrails

۶.۳. مرزبندی سرویس‌ها

upload، lifecycle، signed playback token، session ایجاد، Django/DRF: authorization، tenant scope، audit و state

Media worker جدا: cleanup، probe، validation، transmux/transcode، segment publishing، retry. این کار نباید در web worker یا queue پیش‌فرض notification اجرا شود.

fan-out control—not و **Redis**: playback epoch، progress snapshot، lock، ephemeral health، media bytes

asset/session/permissions/retention پایدار **PostgreSQL**: authority

room/user و short-lived URLها segment و rendition، خصوصی، **Object storage/CDN**: original scoped باشند.

LiveKit: control plane سریع؛ نه ارسال segmentهای ویدئو. Reliable packet برای PLAY/PAUSE/SEEK و disconnect در LiveKit، reliable packet best-effort طبق مستندات lossy heartbeat/telemetry. buffer دائمی ندارد؛ reconnect باید از authority سرور rehydrate شود ([LiveKit Data Packets](#)).

۶.۴. مدل داده پیشنهادی

:SharedVideoAsset

- organization، room/session، uploader، source filename
- upload id، total bytes، committed bytes، checksum
- duration، container، video/audio codec، resolution، fps، bitrate
- status، published duration، available renditions
- storage keys، retention/expiry، failure code
- policy: private/signed، allowed roles، moderation state

وضعیت‌ها:

created → uploading → inspecting → ingesting → playable_partial → processing_variants → ready | failed | cancelled | expired

:SharedPlaybackSession

- active asset و room
- authority participant/user
- state: idle/playing/paused/buffering/ended
- position at epoch، server epoch، playback rate
- sequence/version، updated_at
- strict-classroom یا sync mode: continuous

۶.۵. پروتکل sync

فرمان نمونه:

```
{
  "v": 1,
  "type": "SHARED_VIDEO_COMMAND",
  "sessionId": "svs_123",
```

```

"assetId": "sva_456",
"seq": 42,
"action": "PLAY",
"positionSeconds": 128.45,
"effectiveAtServerMs": 178789000123,
"playbackRate": 1.0
}

```

اصول:

- server یا actor مجاز sequence را معتبر می‌کند؛ frontend visibility مرز امنیت نیست.
- client clock offset باید با ping/response تخمین زده شود؛ sentAt میزبان به‌تنهایی کافی نیست.
- drift کوچک با تغییر rate محدود و کوتاه اصلاح شود؛ drift بزرگ یا پس از reconnect با seek.
- thresholdها از تست UX تعیین شوند؛ مثال آغازین: deadband حدود correction، 250ms، نرخ حدود $\pm 3\%$ ، hard seek حدود 1s.
- هر چند ثانیه snapshot authority منتشر شود؛ join/rejoin از REST/Redis state شروع کند.
- seek فقط تا $\text{seekableEnd} = \text{publishedDuration} - \text{guard}$ مجاز باشد.

۶.۶. سیاست buffer و کاربران ضعیف

دو حالت محصولی:

- **Continuous Sync** (پیش‌فرض): timeline کلاس ادامه دارد؛ کاربر ضعیف کیفیت پایین می‌گیرد و اگر عقب افتاد catch-up می‌کند.
- **Strict Classroom Sync**: اگر درصد مشخصی از کاربران آماده نیستند، host هشدار می‌گیرد و می‌تواند برای همه pause کند.

هیچ viewer ضعیفی نباید خودکار کل کلاس را متوقف کند. گزارش readiness باید aggregate باشد: Ready، At Risk، Buffering، Behind؛ نه فهرست عمومی سرعت اینترنت افراد.

۷. UX پیشنهادی

۷.۱. جریان برگزارکننده

1. «اشتراک ویدئو» را از Tools انتخاب می‌کند.
2. فایل یا URL را وارد می‌کند؛ UI محدودیت codec/مدت/حجم را قبل از upload نشان می‌دهد.
3. preflight محلی resolution، duration، codec، container و احتمال fast path را تشخیص می‌دهد.
4. upload قابل ازسرگیری شروع می‌شود.
5. UI سه نوار یا سه status جدا نشان می‌دهد: upload، پردازش، آماده پخش.
6. Play وقتی فعال می‌شود که startup buffer و rendition حداقلی آماده باشد.
7. timeline بخش آماده، درحال پردازش و unavailable را متفاوت نشان می‌دهد.
8. host حالت sync و دسترسی کنترل را تعیین می‌کند.

۷.۲. جریان بیننده

- ورود دیر هنگام: player با آخرین authority snapshot به موقعیت جاری می‌رود.
- autoplay محدود شده: یک CTA روشن «برای پخش با صدا لمس کنید»؛ هرگز failure خاموش.
- کیفیت Auto به صورت پیش فرض؛ انتخاب دستی 360/720/1080 در صورت موجود بودن.
- volume محلی مستقل از mic؛ audio ducking اختیاری و قابل خاموش کردن.
- هنگام buffer: status شفاف، حفظ آخرین frame و تلاش برای rendition پایین تر.
- اگر frontier ingest تمام شد: «بارگذاری فیلم به این نقطه نرسیده؛ پخش موقتاً مکث شد».

۷.۳. edge case های ضروری

- قطع و resume آپلود؛ refresh یا بستن tab میزبان؛
- فایل خراب، رمزگذاری شده، codec ناشناخته، بدون audio یا چند audio track؛
- upload کندتر از playback؛ worker restart و duplicate chunk؛
- seek به آینده، seek همزمان با processing، پایان فایل پیش از اعلام duration؛
- میزبان disconnect، انتقال host، دو controller همزمان؛
- join دیر هنگام، packet، reconnect، از دست رفته و event تکراری؛
- موبایل، background tab، autoplay، Safari، battery saver؛
- recording جلسه و تعیین اینکه ویدئوی مشترک داخل خروجی recording باشد یا نه؛
- زیرنویس RTL، SRT/VTT، زبان صوتی و accessibility.

۸. امنیت، حریم خصوصی و حقوق

۸.۱. upload و پردازش

- allowlist container/codecs و content sniffing؛ پسوند کافی نیست.
- limit بر track count، bitrate، fps، pixels/frame، duration، bytes و complexity.
- container sandboxed در ffmpeg/ffprobe با CPU/memory/time/file-output limit، شبکه خاموش و filesystem حداقلی.
- original خصوصی، نام storage تصادفی، checksum و scan؛ output تولید شده جدا از source.
- URL امضا شده کوتاه عمر، origin/referrer policy و authorization اتلاق.
- cleanup برای upload های ناقص، orphan های segment و failed jobs.
- quota در سطح tenant و concurrent transcode؛ rate limit برای ساخت upload URL.

۸.۲. محتوا و حقوق

- host باید داشتن حق نمایش/اشتراک را تأیید کند؛ Terms و مسیر report/takedown لازم است.

- retention پیش‌فرض کوتاه و قابل تنظیم؛ «حذف پس از ۲۴ ساعت» نباید بدون نیازسنجی ثابت شود.
- DRM برای MVP ضروری نیست و مانع screen capture کامل نمی‌شود؛ برای محتوای premium تصمیم جدا می‌خواهد.
- محل داده، فروشنده خارجی، sanctions/payment و تعهدات سازمانی باید با مشاور حقوقی و عملیات بررسی شود. این سند مشاوره حقوقی نیست.

۹. مدل هزینه و ظرفیت

متغیرهای اصلی:

- دقیقه ورودی × تعداد rendition ها × ضریب transcode؛
 - دقیقه ذخیره × retention؛
 - دقیقه تماشا × تعداد viewers؛
 - CDN egress/requests؛
 - peak concurrent transcodes و صف؛
 - orphan storage و retry، failure.
- مثال مدل، نه قیمت قطعی: فیلم ۱۲۰ دقیقه‌ای با ۳۰ بیننده اگر همه کامل ببینند، ۳۶۰۰ viewer-minute delivery ایجاد می‌کند. هزینه به فایل ۱ GB محدود نیست؛ watch time و rendition انتخابی مهم‌ترند. prebuffer بیش از حد نیز هزینه delivery ایجاد می‌کند—Cloudflare صریحاً preload و buffering را billable می‌داند ([Cloudflare Pricing](#)).
- Guardrail ها:
- حداکثر مدت، resolution و concurrent transcode بر اساس plan؛
 - retention کوتاه برای فایل جلسه و cold/archive برای موارد ذخیره‌شونده؛
 - 1080p opt-in و 360p-first، 720p default؛
 - cancel job وقتی جلسه پایان یافته و asset دیگر مصرفی ندارد؛
 - داشبورد cost per delivered hour و cost per successful session.
- حد حجم ثابت ۲ GB به‌تنهایی منطقی نیست: دو ساعت ویدئوی ۵ Mbps حدود 4.5GB است. محدودیت بهتر ترکیبی از duration، bitrate، resolution و plan است.

۱۰. نقشه راه پیشنهادی

فاز صفر: Discovery و Spike - حدود ۲ تا ۳ هفته

- جمع‌آوری corpus حداقل 100 فایل واقعی و ناشناس از device/encoder های هدف؛
- اندازه‌گیری VFR، bitrate، moov position، codec، container و pass-through rate؛
- PoC managed در برابر self-hosted؛
- تست شبکه روی ISP های هدف و موبایل/دسکتاپ؛

- تصمیم درباره recording inclusion
- خروجی: benchmark، ADR و go/no-go برای progressive ingest.

فاز یک: Shared Player MVP - حدود ۳ تا ۵ هفته

- entity/session و RBAC
- HLS/MP4 آماده یا URL امضا شده؛
- reconnect، control plane versioned و late join
- volume، host-only control، subtitle، fullscreen پایه؛
- rebuffer، TTFF، analytics و drift
- rollout داخلی و feature flag.

فاز دو: Upload و VOD pipeline - حدود ۴ تا ۷ هفته

- resumable direct upload
- probe/validation/sandbox
- HLS/CMAF 360p/720p و CDN
- quota، retention، retry، lifecycle و cost dashboard
- UX upload/processing/playable
- بار، failure injection و security testing.

فاز سه: Progressive Partial Playback - حدود ۴ تا ۸ هفته پس از Spike موفق

- client preflight و eligible profile
- durable contiguous staging و checkpoint
- incremental segment publish
- playable frontier و seek guard
- throughput predictor و stall prevention
- fallback به upload-complete یا live contribution.

فاز چهار: کیفیت پیشرفته

- caption workflow، multi-audio، audio ducking، VOD→live handoff
- audience readiness و strict sync و moderator transfer
- hardware encoding، 1080p policy و multi-CDN فقط با evidence.
- نه commitment. یک تیم کوچک full-stack بدون media engineer باید بازه بالاتر را مبنا بگیرد.

۱۱. معیارهای موفقیت و SLO آزمایشی

محصول

- درصد جلسات واجد شرایط که shared video را با موفقیت شروع می‌کنند؛
- زمان از انتخاب فایل تا اولین frame؛
- درصد uploadهای resume شده؛
- completion rate پخش و abandon پیش از شروع؛
- CSAT برگزارکننده و بیننده.

کیفیت تجربه

- TTFF p50/p95؛
- rebuffer ratio و rebuffer count per viewer-hour؛
- drift p50/p95 و زمان recovery؛
- late-join catch-up success؛
- درصد زمان روی 360/720/1080؛
- درصد viewers با startup failure.

عملیات

- transcode real-time factor؛
- orphan rate و queue wait p95، job failure/retry؛
- storage/CDN cost per delivered hour؛
- concurrent capacity و CPU/GPU utilization؛
- signed URL/auth failure و abuse events؛

SLO پیشنهادی فقط برای Pilot و پس از baseline تعیین شود. مثال آزمایشی: TTFF کمتر از 8 ثانیه برای asset آماده، drift p95 کمتر از 500ms در شبکه پایدار، rebuffer ratio کمتر از 1٪ برای viewerهایی که rendition مناسب دارند. این اعداد هدف اولیه‌اند، نه وعده بازار.

۱۲. تست و پذیرش

ماتریس فایل

- MP4 faststart و moov-end، WebM، MOV و MKV؛
- audio-only و H.264/AAC، HEVC، VP9، AV1، PCM؛
- 5 دقیقه، 30 دقیقه و 120 دقیقه؛
- 240p تا 4K، VFR/CFR، portrait/landscape؛

- فایل خراب، encrypted، truncated و metadata سنگین.

ماتریس شبکه

- upload میزبان 256kbps تا 20Mbps با loss/jitter؛
- viewerهای ناهمگون 2G/3G/4G/ADSL/fiber؛
- قطع 5/30/120 ثانیه و resume؛
- slow origin، CDN cache miss/hit و 5xx؛
- 1، 10، 50 و 100 viewer همزمان.

پذیرش حیاتی

- هیچ segment ناقصی advertise نشود؛
- seek فراتر از frontier رد و UX توضیح داده شود؛
- duplicate/reordered command idempotent باشد؛
- reconnect state درست را بازیابی کند؛
- participant بدون مجوز نتواند start/seek/stop کند؛
- cross-tenant asset و URL قابل دسترسی نباشد؛
- پایان جلسه upload/transcode را طبق policy cancel/retain کند؛
- object URL، listener، timer و player buffer cleanup شود.

۱۳. تصمیم‌های لازم پیش از ساخت

1. اولویت واقعی کدام است: شروع فوری، بالاترین کیفیت، یا کمترین هزینه؟
2. آیا فایل باید در recording نهایی دیده شود؟
3. retention پیش‌فرض و plan quota چیست؟
4. managed vendor از نظر شبکه، پرداخت و قرارداد قابل استفاده است؟
5. MVP فقط desktop است یا mobile upload نیز لازم است؟
6. codec/profile پشتیبانی‌شده و max duration/resolution چیست؟
7. در شبکه ضعیف timeline ادامه یابد یا کلاس برای همه pause شود؟

۱۴. توصیه نهایی

Go، با scope کنترل‌شده. مسئله واقعی و تمایزپذیر است، ولی ساخت «progressive ingest» عمومی برای هر فایل» در MVP ریسک غیرضروری دارد. ابتدا shared player و VOD آماده را عرضه کنید، همزمان Spike progressive را با corpus واقعی انجام دهید. اگر شروع فوری حیاتی است، live contribution موقت را اضافه کنید و پس از ready شدن rendition به HLS handoff دهید.

به جای شعار «بدون لگ»، روی سه نتیجه قابل اندازه گیری بفروشید: شروع زودتر، کیفیت مستقل برای هر بیننده، و کنترل همگام برای مدرس.

ضمیمه B - منابع کلیدی

- W3C، [ISO BMFF Byte Stream Format](#)، 2024
- IETF، [RFC 8216: HTTP Live Streaming](#)، 2017
- Apple، [HLS Authoring Specification](#)، جاری.
- FFmpeg Project، [Formats Documentation](#)، جاری.
- tus.io، [Resumable Upload Protocol](#)، 1.0.0
- AWS، [Multipart Upload Overview](#)، جاری.
- LiveKit، [Data Packets](#)، جاری.
- Zoom، [Sharing a recorded video with sound](#)، جاری.
- BigBlueButton، [Share an External Audio/Video Link](#)، جاری.
- Adobe Connect، [Guidelines for sharing MP4 files](#)، 2025-03-05
- Cloudflare، [Stream Uploads](#) و [Pricing](#)، 2026
- Mux، [Mux Uploader](#) و [Pricing](#)، جاری.

محدودیت های تحقیق

- این تحقیق مستندات رسمی و کد پروژه را بررسی کرده و benchmark اجرایی روی اینترنت کاربران ایران انجام نداده است.
- نبود سند رسمی برای play-before-upload-complete در رقبا، اثبات قطعی نبود قابلیت در تمام نسخه ها نیست.
- قیمت، availability منطقه ای و شرایط فروشندگان متغیر است و پیش از قرارداد باید دوباره راستی آزمایی شود.
- برآورد زمان بدون Spike و اندازه تیم، تخمینی است.
- توصیه حقوقی ارائه نشده است.

ثبت جستوجو و معیار توقف

جستوجو در منابع رسمی W3C، IETF، Apple، FFmpeg، tus، AWS، LiveKit، Zoom، Microsoft، Google، Cloudflare، Teleparty، Watch2Gether، Adobe، BigBlueButton و Mux انجام شد. موج دوم مشخصاً ادعاهای moov، S3/tus، LL-HLS، codec copy، readiness، seek، telemetry، corpus و شکاف های باقی مانده فقط با benchmark عملی قابل پاسخ بود. تمام ادعاهای consequential منبع اولیه یا محدودیت صریح داشتند و شکاف های باقی مانده فقط با benchmark عملی قابل پاسخ بود.